Enhancing TIAGo Robot Capabilities: Introducing Azure Kinect DK for Advanced Grasping of Kitchen Objects

Ramírez, H., Muñoz, L. A.
Tec de Monterrey, School of Engineering and Sciences, Nuevo León, Mexico
Contacts: ha.ramirezv@outlook.com ; amunoz@tec.mx

***Abstract*—This research project illustrates robotic manipulation through the integration of visual markers, the Azure Kinect DK sensor, and the TIAGo robot. It addresses challenges in marker detection accuracy, robot hand pose estimation, and efficient object manipulation to open and close a drawer. The project showcases real-time algorithm capabilities with improved marker detection and consistent pose predictions. Reducing time execution.**

***Index Terms*—Azure Kinect DK, april tags, TIAGo**

I. Introduction

In the dynamic field of robotics, the quest for precise object manipulation continues to propel technological and methodological advancements. This document delves into the complexities of opening and closing a dishwasher drawer, achieved through a collaborative effort leveraging the capabilities of Azure Kinect DK and the utilization of visual markers for Hand Tracking.

At the core of the project lies the integration of Azure Kinect DK, visual markers, and the TIAGo robot. These technologies are harnessed to predict the pose of the robot hand and target objects, facilitating object manipulation. The project unfolds in several stages, including pose estimation, visual servoing, and motion planning.

Through rigorous simulations and experiments, the robustness and efficacy of the implemented methodologies are assessed. This document provides a comprehensive overview of the project's stages.

II. State of the art

A. Visual Fiducial system

Visual fiducials [1] are artificial markers created to facilitate their straightforward identification from each other. A visual fiducial was designed to enable its automatic detection and precise localization even when it is presented in low-resolution scenarios. They have different applications, for example, QR codes are designed to be detected with high resolutions cameras allowing the access to hundreds of bytes, on the other hand, a visual fiducial has 12 bits of information. Also the position of this systems is not important, because they can provide camera-relative position and orientation of a tag, also they can have virtual reality applications. This systems are very helpful to generate robot trajectories and close loop controllers. These types of systems have also found application in automatic object calibration, aiding in the determination of a work object's frame [2].

a) ARToolKit [3]: One of the early tag tracking systems in augmented reality (AR) applications. It overlays digital content on real-world objects by recognizing physical markers or 'tags'. The system doesn't directly encode information in binary but compares images with different patterns in a database of known tags.

b) ARTag [4]: Inspired on ARToolKit system. ARTag improves reliability and decrease light conditions effect. Also solve false positive problem presented on a previous version. ARTag encodes information on 36 bits, first 10 for id, and the remaining to correct errors from redundancy.

c) ARToolKitPlus [5]: Originally based on ARToolKit, this system represents a significant improvement over its predecessors. It introduces binary marker patterns for encoding marker IDs (as ARTag system), resulting in vastly improved performance compared to the previous versions, as binary pattern recognition is faster than template markers.

d) AprilTag [6]: Improves previous versions, incorporating fast and robust line detection system, this systems integrates a graph-based image segmentation algorithm that allows lines to be precisely estimated. Its uses can be include on camera calibration, mobile robotics and augmented reality.

An illustrative case of its use for visual feedback can be seen in the work involving LARICS markers [7]. This project demonstrated a self-localization system for a mobile robot, enabling it to serve as a navigation guide for autonomous movement along a predefined path.

Visual fiducials are a highly valuable feature for object tracking as they enable precise pose estimation. However, it's important to emphasize the significance of RGB-D cameras in this estimation task for a better accuracy of data.

B. Depth sensors

Depth sensors are a crucial component in computer vision applications, utilized in fields ranging from manufacturing to robotics and autonomous vehicles. They play a significant role by introducing a third dimension to digital imaging, thus facilitating the replication of human vision [8]. These sensors create a depth map in which each pixel contains a numerical value representing the distance from the sensor to an object. This capability empowers machines to gain

a more comprehensive understanding of and navigate their surrounding environment.

There are various technologies for obtaining depth information, including:

- *Structured and Coded Light*: This method involves comparing a known projected pattern with the deformed pattern resulting from the projected light.
- *Stereoscopy*: Inspired by human vision, this approach employs a set of sensors to create depth information. Stereoscopy matches corresponding points from two closely positioned cameras using epipolar geometry to derive depth information.
- *Time-of-Flight (ToF)/LiDAR*: This technology measures the time delay between a light source pulse and the received reflected light. It is the most commonly used technology in depth sensors due to its simplicity and ease of use.

As examples of sensors that use these technologies are the Kinect v1/v2 (ToF), Azure Kinect DK (ToF), Zed 2i (Stereolabs), among others. After conducting experiments to measure their accuracy, the results indicated that Azure Kinect DK achieved the highest accuracy at 96%, followed by Kinect v2 at 90%, and Zed 2i at 89%. This assessment was based on the evaluation of skeleton data generated by their respective depth maps [9]. Other experiment [10] showed that Azure Kinect DK performs better in terms of accuracy compared with its past kinect versions.

In summary, the combination of visual fiducial systems and depth sensors offers a powerful set of tools for robotics and computer vision. These technologies have a variety of applications, autonomous robot navigation, augmented reality, object tracking, or relative visual servoing as an example.

III. Aims and Scope

This papper aims to enhance visual markers detection using AprilTag library, Azure Kinect DK sensor and the TIAGo robot to detect and manipulate objects.

1) Predict the pose hand of the robot and the target using AprilTags and Azure Kinect DK sensor.
2) Grasp kitchen objects with the hand of the robot.

IV. Technical Infrastructure

The hardware includes the Sensor Azure Kinect DK, the TIAGo Robot, and Visual Markers. On the software front, we employ Ros (Robot Operating System), the AprilTag Library, and Docker/Ubuntu for seamless integration and efficient functioning.

V. Methodology

A. AprilTag Pose Estimation

To begin with, this stage implements the AprilTag ROS wrapper [11], enabling compatibility between AprilTag and the ROS environment, and the Azure Kinect DK Ros driver [12], allowing the use of Azure Kinect DK in ROS either. Due to the fact that the AprilTag wrapper does not have a depth image input, only RGB image and camera calibration as an input topics, its implementation with any data processing results on a low pose estimation. Because of this, an algorithm to enhance accuracy of pose detection was developed.

1) Azure Kinect DK configuration: The sensor engages in intricate data processing, performing comprehensive tasks such as complete point cloud reconstruction and depth information analysis. However, this intricate processing introduces a latency of 5Hz. To address this, the configuration of the Azure Kinect DK has been modified. Instead of the original point cloud reconstruction and depth analysis, the sensor now delivers RGB images and depth information at an enhanced frequency of 30 frames per second (fps). This adjustment optimizes real-time performance, ensuring a smoother and more responsive experience.

2) AprilTag pose enhanced: The tag pose estimation was made by processing the pixel information of each corner and center of the tag.

a) Tag position estimation: The AprilTag ROS wrapper was implemented to retrieve the corner pixels of each tag. In combination with the RGB and depth images from the Azure Kinect DK driver, it facilitated the determination of the depth for each corner and enabled the estimation of the depth at the center of the tag.

Due to latency arising from the complete point cloud reconstruction of the x and y camera coordinates required for obtaining the full position of the tag, a solution was implemented. Instead of processing the entire point cloud, a partial reconstruction was adopted for each corner and the center of the tag. This modification effectively reduces latency in data estimation. To achieve this, knowledge of intrinsic values from the camera matrix was crucial, and these values were obtained through the camera calibration process using the Matlab calibrator tool.

The camera matrix was a concept integrated in this partial reconstruction. The projection camera matrix transforms 3D world coordinates into a 2D camera coordinates, this matrix uses the intrinsic values (transforms from camera system coordinates to camera pixel coordinates) and the extrinsic values (transforms world coordinates into camera coordinates). This matrix is shown on equation (1), where u and v are the pixel coordinates; x, y, z are the coordinates in the camera system; f_x, f_y, c_x, c_y are the intrinsic properties; and r and t represent the rotation and translation matrices.

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (1)$$

We're not interested in the extrinsic values from the camera in this paper, because we want to design an algorithm that can be able to adapt to any position of the camera. Also we want to use the camera frame to estimate pose, in this way the position of the camera is not important while the hand of the robot and the target are viawable in the Azure Kinect DK.

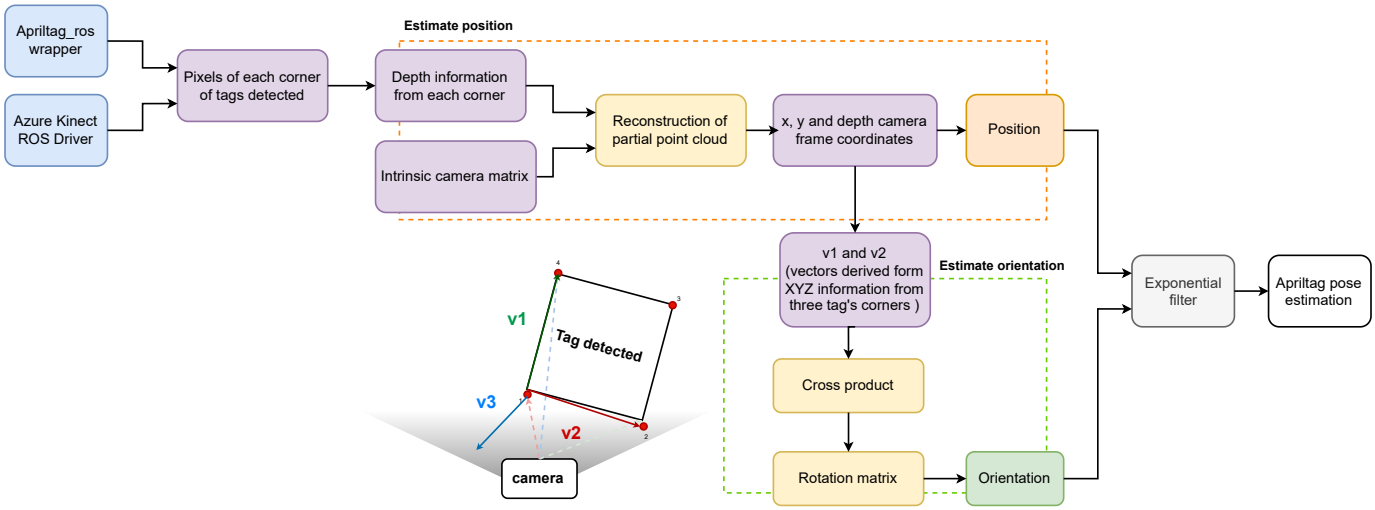

Fig. 1. AprilTag pose estimation improvement

With this in mind, then our camera matrix is expressed on equation (2).

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{2}$$

Equation (2) was used for a partial point cloud reconstruction, using pixels and depth information, obtaining the camera coordinates of each corner and center of the tag, (3) y (4).

$$x = \frac{z(u-cx)}{fx} (3) \qquad y = \frac{z(v-cy)}{fy} (4)$$

b) Tag Orientation Estimation: The previous position coordinates from each tag corner were use to improve orientation prediction. This coordinates allowed to obtain two vectors of two sides of the tag on the camera frame $v1$ and $v2$, a third vector $v3$ was obtained by using the cross product, as it is show in (5), (6) and (7). Where $v1$, $v2$, $v3$ are vectors, $o1$, $o2$, $o3$ are vectors from the origin of the camera (o) to each tag corner (1, 2, 3). Also figure 1 can help to understand this process.

$$\overrightarrow{v1} = \overrightarrow{o1} - \overrightarrow{o4} \tag{5}$$

$$\overrightarrow{v2} = \overrightarrow{o1} - \overrightarrow{o2} \tag{6}$$

$$\overrightarrow{v3} = \overrightarrow{v1} \times \overrightarrow{v2} \tag{7}$$

This three vectors $v1$, $v2$ and $v3$ indicates the tag rotation, (8), which improves orientation.

$$rotationMatrix = \begin{bmatrix} v1_x & v2_x & v3_x \\ v1_y & v2_y & v3_y \\ v1_z & v2_z & v3_z \end{bmatrix} \tag{8}$$

c) Low pass Filter: Finally, a low pass filter was implemented because a fast movement of the tag can produce a wrong pose estimation. This filter makes a new prediction using the previous prediction and the current value from the detection (9).

$$y(k) = \alpha \cdot y(k-1) + (1-\alpha) \cdot x(k) \tag{9}$$

Where, $x(k)$ is the input value at time k, $y(k)$ is the filter output at time k, $y(k-1)$ is the preveous predicted value at time k, α is a constant between 0 and 1 which uses to have values between 0.8 and 0.9. Generally, the pose estimation improvement process is explain in figure 1.

B. Robot Hand pose estimation

The previous tag detection was used to estimate the robot hand pose. Four visual tags were used to build a square around the hand of the robot which allows to predict position and orientation of the hand as it is shown in figure 2. This algorithm prioritize the detection of the smallest id, also orientation and position changes according with the id detected. For this the concept of transformation matrices was integrated. This process allowed to find the pose of the center of the hand, giving hand position and orientation coherence according to the tag detected.

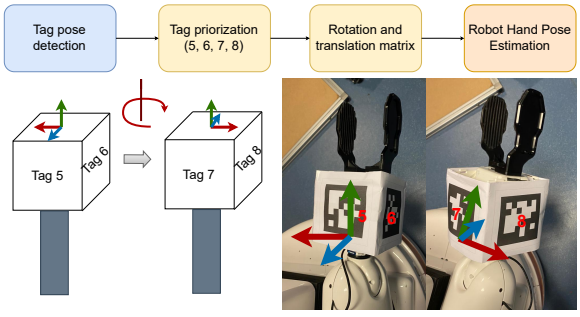

Fig. 2. Robot hand pose estimation

C. Controller: Relative Visual Servoing

Position and orientation controllers were implemented to obtain hand velocity. To begin with, a proportional saturated controller (P controller) was designed for both orientation and position. A relative transformation matrix was implemented to calculate the error between the hand pose and the target, encompassing both rotation (10) and position (11) errors. As it is shown in figure 3.

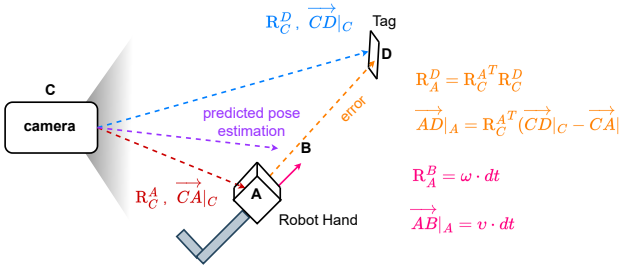

Fig. 3. Control diagram for position and orientation

$$RotationError = R_C^{A^T} R_C^D \tag{10}$$

$$PositionError = R_C^{A^T}(\overrightarrow{CD}|_C - \overrightarrow{CA}|_C) \tag{11}$$

Angular (12) and linear (13) velocities were obtained by an simple P controller implementation.

$$\omega = kp_\omega \cdot RotationError \tag{12}$$

$$v = kp_v \cdot PositionError \tag{13}$$

For feedback, a complementary filter was designed to predict the next position and orientation of the hand. To achieve this, angular and linear velocity were integrated into the hand's pose to calculate the subsequent position and orientation. This integration was performed in conjunction with the actual hand position and orientation obtained from the Azure Kinect DK.

1) Predicted pose estimation: Additionally, the velocity integration into the hand pose was performed using a relative transformation matrix, as illustrated in Figure 3. The integration process resulted in a new predicted pose estimation, providing updated position and orientation values in regarding the previous state which is in the hand frame. Equations (14) and (15) shows pose on the robot hand frame, while pose on the camera frame is expressed by equations (16) and (17) Pose on robot hand frame:

$$R_A^B = \omega \cdot dt \tag{14}$$

$$\overrightarrow{AB}|_A = v \cdot dt \tag{15}$$

Pose on camera frame:

$$R_C^B = R_C^A R_A^B \tag{16}$$

$$\overrightarrow{CB}|_C = \overrightarrow{CA}|_C + R_C^A \overrightarrow{AB}|_A \tag{17}$$

2) Complementary filter: A complementary filter for position and orientation was designed to predict the following pose using the measured pose and the predicted pose estimation which improves robot hand pose estimation with the calculated and detected pose. This filter is a steady-state Kalman filter (wiener filter) [13] used to filter out noise from a signal. This filter is expressed by equation (18). Where, x and y are noisy measurements of a signal z; $\widehat{z}$ is the estimate produced by the filter: α is a constant between 0 and 1.

$$\widehat{z} = x \cdot (1 - \alpha) + y \cdot \alpha \tag{18}$$

The control scheme is presented in figure 4, where a controller diagram is shown.

D. Object Manipulation

A state machine has been designed to execute the object manipulation task. The state machine are intricately crafted for efficient object manipulation, employing a different stages for the motion planning process.

On the other hand, this state machine facilitates the hand's ability to reach the object while simultaneously avoiding potential collisions. Two target points guide the process: the first aims to preemptively prevent future collisions, and the second is geared toward successfully reaching the object. In this latter phase, the hand is capable of closing when the object is in proximity. The third stage enables the system to grasp and manipulate the object seamlessly.

This process is shown on figure 5, in which every stage of the machine is explained. This diagram explained how the robot is able to manipulate the drawer, once the hand achieve the first target then it move to the second target to grasp the object and manipulate it.

VI. Simulations and Experiments

A. Visual Marker detection improvement

A simulation was created to visualize data both before and after the implementation of improvements. The results are presented in the following video: Stage 1, Visual Marker Detection Improvement.

In the video, the left side demonstrates a simulation with low data accuracy, operating at a frequency of 5 Hz. Conversely, the right side showcases the detection improvement, now operating at an enhanced frequency of 30 Hz.

B. RobotHand pose estimation

For the robot hand pose stage, we conducted simulations to validate the consistency in position and orientation prediction. In these simulations, each tag was presented to the camera. Using the detected tag, rotation and translation matrices were applied to the original tag pose, ensuring coherence in the predicted robot hand pose. The results are visually represented in Figure 6 and can be further explored in the accompanying video: Stage 2, Robot Hand Pose Simulation.

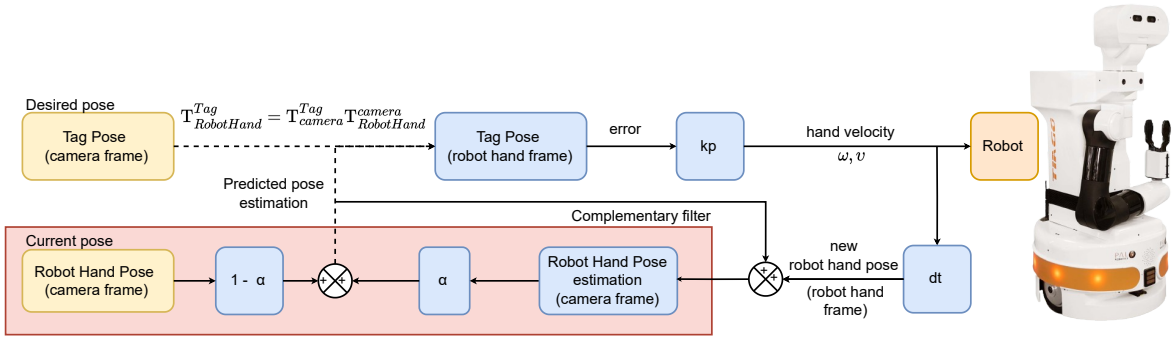

Fig. 4. Control scheme implemented for the object manipulation

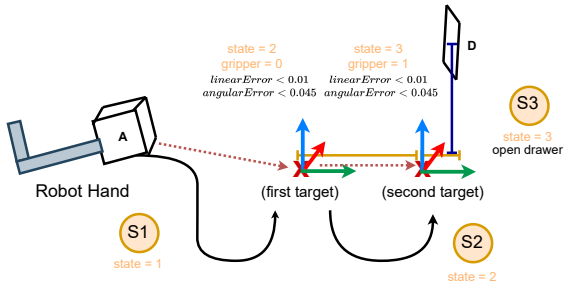

Fig. 5. State machine to manipulate objects

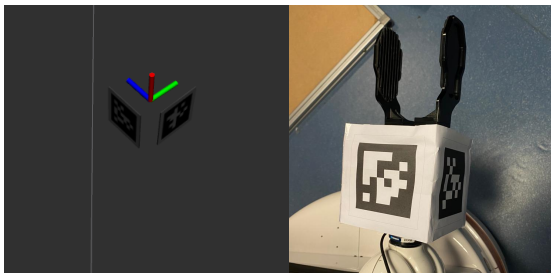

Fig. 6. Robot hand pose estimation which results from the tag detection.

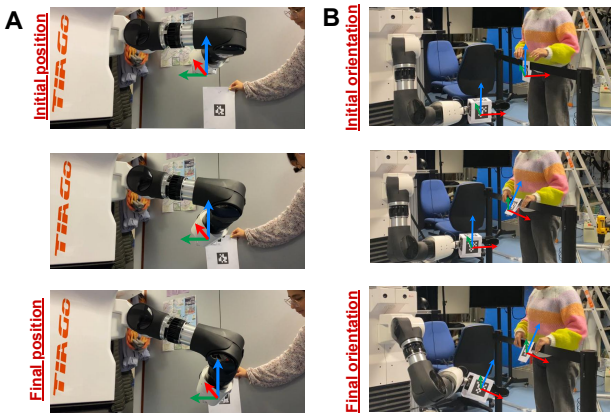

Fig. 7. Pose controller experiments in TIAGo robot. "A" shows position experiment while "B" shows the orientation.

C. Relative Visual Servoing

The controller design was executed in two stages: initially focusing on the position controller, followed by the orientation controller. These experiments were conducted using the TIAGo robot, and this experiments are ilustrated in figure 7.

1) Controller for position: The experiments depict the hand starting at an initial position, and through the controller, the hand successfully reaches the desired target position aligned with the visual tag detected, as it is shown on figure 7, A.

2) Controller for orientation: For orientation controller, at the beginning the robot starts on an initial orientation that can vary, after the controller the hand can reach the orientation that the target has, as it is shown on figure 7, B.

D. Object Manipulation

In this stage, a bottle and a dishwasher drawer were implemented for experiments.

The first experiment involved grasping a bottle from a surface. A simple three-state machine was employed for this task. The states included collision avoidance, reaching the object, and manipulating the bottle. Subsequent experiments involved using a dishwasher drawer for tasks such as opening and closing. The state machine for these tasks was modified to incorporate additional target points, ensuring efficient execution without collisions. For opening the drawer, the state machine employed five states to navigate to open it, move away from the drawer, and return the hand. This entire process is illustrated in Figure 8. Conversely, for closing the drawer, a different set of five states was employed to accomplish the task seamlessly. Figure 9 presents the experiment, highlighting the designed states to prevent collisions and return the hand to the initial position upon task completion. All this experiments are available in videos: bottle experiment, and dishwasher experiments.

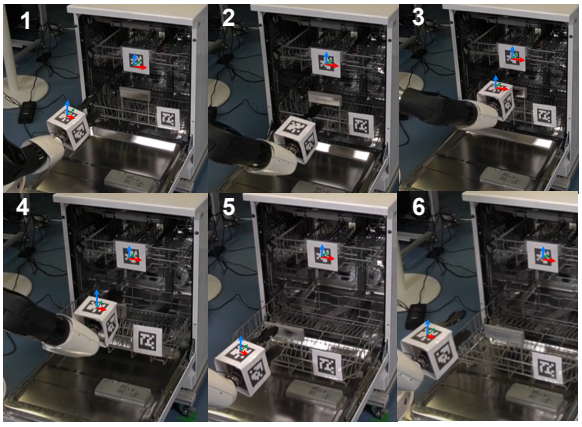

Fig. 8. Opening dishwasher drawer.

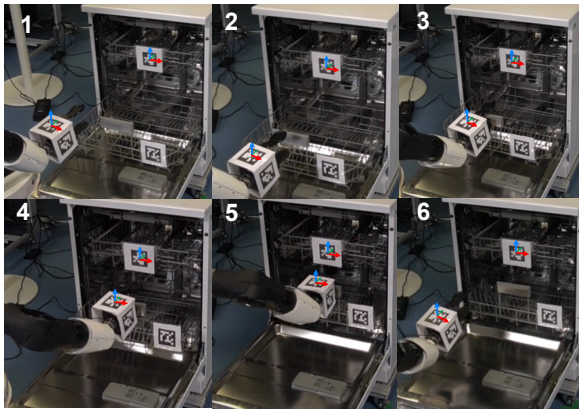

Fig. 9. Closing dishwasher drawer.

VII. Results

The object manipulation experiments demonstrate the effectiveness of the designed algorithm, which improves the accuracy of marker detection when combined with the controller. This combination successfully accomplishes various tasks, such as grasping a bottle or opening and closing a dishwasher drawer. On the other hand, grasping a bottle proves to be a relatively straightforward task compared to the manipulation of the drawer. Opening the drawer poses a more challenging task to achieve than closing it. In the process of opening the drawer, experiments required human intervention to assist the robot in achieving successfully the task. This necessity arose due to the dependence on the initial position of the robot, where the success of opening the drawer with the correct orientation was contingent on a small guidance to the drawer at the end when the hand leave the drawer. To close the drawer is easier for the hand, because the hand only applies a small effort with the gripper as a guide to put the drawer into the washing machine without closing the gripper. Also, this experiments show that this algorithm can be apply to manipulate different objects, from manipulating objects as simple as a bottle to manipulating more complex objects such as drawers.

VIII. Conclusion

This project presents a comprehensive approach to robotic object manipulation, combining advanced technologies such as visual markers, depth sensors, and a robust control system. The project successfully addresses challenges in marker detection accuracy, robot hand pose estimation, and efficient object manipulation. The enhancement in visual marker detection, illustrated in simulations, demonstrates the algorithm's capability to operate in real-time applications. The experiments validate the consistency and coherence of the predicted poses based on visual markers. The experiments in object manipulation, involving grasping a bottle and manipulating a dishwasher drawer, underscore the algorithm's adaptability to different scenarios and objects. While challenges persist, particularly in the orientation control during the drawer manipulation, the overall results showcase the potential of the RoboHand Navigator project to contribute to advancements in robotic object manipulation.

Acknowledgement

This work was completed with the support and guidance of Serena Ivaldi, Jean Baptiste, and Quentin Rouxel from INRIA in Nancy, France.

References

[1] Z. Zhuming, H. Yongtao, Y. Guoxing, and D. Jingwen. "DeepTag: A general Framework for Fiducial Marker Design and Detection". IEEE Transitions on pattern analysis and machine intelligence, vol. 45, no. 03, pp. 2931-2944, 2023.

[2] S. Bernard and W. Lihu. "Automatic work objects calibration via a global–local camera system", Robotics and Computer-Integrated Manufacturing, vol. 30, pp. 673-683, December, 2014.

[3] K. Hirokazy and B. Mark. "Marker Tracking and HMD Calibration for a Video-based Augmented Reality Conferencing System", Proceedings 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR'99), pp. 85-94, 1999.

[4] S. Ksenia, S. Artur, L. Hongbing and M. Evgeni. "Comparing Fiducial Marker Systems Occlusion Resilience through a Robot Eye", pp. 273-278, June, 2017.

[5] W. Daniel and S. Dieter. "ARToolKitPlus for Pose Tracking on Mobile Devices". 2007.

[6] O. Edwin. "AprilTag: A robust and flexible visual fiducial system.", 2011 IEEE International Conference on Robotics and Automation, pp. 3400-3407, May 2011.

[7] M. Alan, M. Damjan and D. Ivica. "A low cost vision based localization system using fiducial markers", IFAC Proceedings Volumes, no. 41:2, pp. 9528-9533, 2008.

[8] H. Raphaël, S. Quentin, S. Edouard, G. Pierre and T. Antoine. "Evaluation of Low-cost depth sensors for outdoor applications", 7th International Workshop LowCost 3D - Sensors, Algorithms, pp. 101-108, 2022.

[9] S. Violeta and S. Angela. "Evaluating Automatic body Orientations Detection for Indoor Location from Skeleton Tracking Data to Detect Socially Occupied Spaces Using the Kinect v2, Azure Kinect DK and Zed 2i", Sensors, no. 22, pp. 3798.

[10] K. Gregorij. "Evaluating the Accuracy of the Azure Kinect DK and Kinect v2." Sensors (Basel, Switzerland) no. 22,7 pp. 2469. March 2022.

[11] ROS. "apriltag ros" Ros Wiki, https://wiki.ros.org/apriltag_ros, 2022.

[12] Microsoft. Azure Kinect DK ROS Driver Usage, 2019.

[13] T. Walter. "A Comparison of Complementary and Kalman Filtering", IEEE Transactions on Aerospace and Electronic Systems, vol. AES-11, no. 3, pp. 321-325, May 1975.

Resources

- Repository.
- Slides.
- Video Presentation.
- Experiments.